



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт Информационных технологий

Кафедра математического обеспечения и стандартизации информационных
технологий

Отчет по практической работе №3

по дисциплине «Тестирование и верификация программного обеспечения»

Выполнили:

Студенты группы ИКБО-32-23

Мингазутдинов А.Р.

Команда «ВМР»

Ильичев Г.П.

Проверил:

Отчет представлен

«__»_____2025г.

Москва 2025

1. Цель и задачи практической работы

Цель работы состоит в изучении и применении различных подходов к разработке программного обеспечения, основанного на тестировании, для повышения качества, надёжности и поддерживаемости кода.

Для достижения поставленной цели работы студентам необходимо выполнить ряд задач:

- 1) изучить теоретические основы методологий тестирования: TDD, ATDD, BDD и SDD;
- 2) исследовать преимущества и недостатки каждого подхода;
- 3) реализовать практические примеры для каждого метода;
- 4) проанализировать влияние интеграции тестирования на архитектуру и качество программного продукта;
- 5) подготовить итоговый отчёт с выводами и рекомендациями по интеграции подходов в современные процессы разработки.

2. Описание работы

Практическая работа выполняется индивидуально.

В процессе выполнения необходимо разработать программный продукт под Вашим номером, используя TDD, ATDD, BDD и SDD. В отчете необходимо подробно описать все этапы разработки, представленные выше в примерах.

85. Приложение для флэш-карт (изучение языков)

Функции — создание карточек, запуск сессии, отслеживание результатов.

TDD — тесты для создания и обновления карточек.

ATDD — сценарии для проверки эффективности обучения.

BDD — сценарий «Given карточки созданы, When прохожу сессию, Then вижу статистику».

SDD — таблицы с примерами карточек и результатами тестирования

3. Приложение для флэш-карт (изучение языков)

1. TDD (Test-Driven Development) — Разработка через тестирование

Суть метода:

В этом подходе разработчик **сначала пишет тесты**, а уже потом — сам код программы, чтобы эти тесты проходили успешно.

Простыми словами:

Сначала мы формулируем, **что программа должна уметь делать** (в виде тестов),

а потом пишем минимальный код, который заставляет тесты пройти.

Процесс:

1. Пишется тест, проверяющий, что при добавлении карточки со словом `cat` и переводом `кот`
2. она действительно появляется в списке карточек.
3. Запускаем тест — он падает, потому что функция `add_flashcard()` ещё не реализована.
4. Реализуем функцию добавления карточек.
5. Запускаем тест повторно — теперь он проходит успешно.
6. После этого можно улучшить код, не боясь поломать логику — тесты всё контролируют.

Зачем:

Такой подход помогает писать **чистый и надёжный код**, потому что каждая функция сразу проверяется тестом.

```
TDD.py > ...
1  import unittest
2  from flashcards_app import FlashcardApp
3  from io import StringIO
4  from unittest.mock import patch
5
6  class TestFlashcardApp(unittest.TestCase):
7
8      def setUp(self):
9          # Этот метод запускается перед каждым тестом.
10         # Он создаёт новый экземпляр нашего приложения, чтобы тесты не влияли друг на друга.
11         self.app = FlashcardApp()
12
13     def test_add_flashcard(self):
14         # Проверяем, что карточка добавляется корректно.
15         self.app.add_flashcard("cat", "кошка")
16         # assertIn проверяет, есть ли элемент в списке карточек.
17         self.assertIn(("cat", "кошка"), self.app.flashcards)
18
19     def test_update_flashcard(self):
20         # Проверяем, что можно изменить перевод существующей карточки.
21         self.app.add_flashcard("dog", "пёс")
22         self.app.update_flashcard("dog", "собака")
23         self.assertIn(("dog", "собака"), self.app.flashcards)
24
25     def test_flashcard_not_found(self):
26         # Проверяем, что при попытке обновить несуществующее слово выбрасывается ошибка.
27         with patch("sys.stdout", new=StringIO()) as fake_out:
28             self.app.update_flashcard("bird", "птица")
29             output = fake_out.getvalue().strip()
30             self.assertEqual(output, "Карточка 'bird' не найдена.")
31
32     if __name__ == '__main__':
33         unittest.main()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
> & C:/Users/artem/AppData/Local/Programs/Python/Python313/python.exe c:/Users/artem/Desktop/prog/TDD.py
Карточка 'cat' -> 'кошка' добавлена.
..Карточка 'dog' -> 'пёс' добавлена.
Перевод для 'dog' обновлён на 'собака'.
.
-----
Ran 3 tests in 0.001s

OK
PS C:\Users\artem\Desktop\prog> |
```

Фото 1 — метод TDD

2. ATDD (Acceptance Test-Driven Development) — Разработка через приёмочные тесты

Суть метода:

Тесты создаются **вместе с заказчиком или пользователем**.

Они описывают, **как система должна работать с точки зрения клиента**.

Простыми словами:

Перед началом работы все договариваются, **что считается правильной работой программы**.

Эти договорённости превращаются в тесты.

Пример:

Если пользователь добавляет карточку со словом *sun* и переводом *солнце*, она должна появиться в списке карточек.

Если пользователь проходит тренировку и правильно вводит все переводы,

приложение должно вывести сообщение:

«Сессия завершена. Правильных: 2, Неправильных: 0»

→ Эти проверки превращаются в приёмочные тесты, и пока приложение не проходит их, работа не считается завершённой.

Зачем:

ATDD помогает избежать недопонимания между разработчиками и заказчиком,

так как требования сразу выражены в виде проверяемых тестов.

```
ATDD.py > TestFlashcardApp_ATDD
1 from flashcards_app import FlashcardApp
2 import unittest
3 from unittest.mock import patch
4 from io import StringIO
5
6 class TestFlashcardApp_ATDD(unittest.TestCase):
7
8     def setUp(self):
9         self.app = FlashcardApp()
10
11     @patch("builtins.input", side_effect=["солнце", "луна"])
12     def test1(self, mock_input):
13         # Шаг 1. Добавляем карточки
14         self.app.add_flashcard("sun", "солнце")
15         self.app.add_flashcard("moon", "луна")
16
17         # Шаг 2. Запускаем тренировку (ответы подставляются автоматически)
18         with patch("sys.stdout", new=StringIO()) as fake_out:
19             result = self.app.start_session()
20             output = fake_out.getvalue()
21
22         # Шаг 3. Проверяем результат
23         self.assertEqual(mock_input.call_count, 2)
24         self.assertEqual(output.count("Верно!"), 2)
25         self.assertNotIn("Неверно", output)
26
27     @patch("builtins.input", side_effect=["кошка", "неc"])
28     def test2(self, mock_input):
29         self.app.add_flashcard("cat", "кошка")
30         self.app.add_flashcard("dog", "собака")
31
32         with patch("sys.stdout", new=StringIO()) as fake_out:
33             self.app.start_session()
34             output = fake_out.getvalue()
35
36         # Проверим, что пользователь дал 2 ответа
37         self.assertEqual(mock_input.call_count, 2)
38         self.assertEqual(output.count("Верно!"), 1)
39         self.assertEqual(output.count("Неверно."), 1)
40
41 if __name__ == "__main__":
42     unittest.main()
43
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\artem\Desktop\prog> & C:/Users/artem/AppData/Local/Programs/Python/Python313/python.exe c:/Users/artem/Desktop/prog/ATDD.py
Карточка 'sun' -> 'солнце' добавлена.
Карточка 'moon' -> 'луна' добавлена.
.Карточка 'cat' -> 'кошка' добавлена.
Карточка 'dog' -> 'собака' добавлена.
.
-----
Ran 2 tests in 0.001s

OK
PS C:\Users\artem\Desktop\prog> 
```

Фото 2 — метод ATDD

3. BDD (Behavior-Driven Development) — Разработка через поведение

Суть метода:

Требования к программе описываются **на простом человеческом языке** (например, английском), в формате сценариев «**Given–When–Then**» (Дано–Когда–Тогда).

Простыми словами:

Мы описываем **поведение системы** так, как его видит пользователь.

Эти описания можно автоматически превратить в тесты.

Пример:

Feature: Проведение тренировки с карточками

Scenario: Пользователь правильно отвечает на все слова

Given карточки "sun" -> "солнце" и "moon" -> "луна" добавлены в приложение

When пользователь проходит сессию и вводит правильные переводы

Then приложение сообщает "Сессия завершена. Правильных: 2,
Неправильных: 0"

Зачем:

BDD объединяет разработчиков, тестировщиков и заказчиков, так как сценарии понятны всем и могут автоматически проверяться при запуске тестов.


```
features > ≡ flashcards.feature
  1 Feature: Learning flashcards
  2   In order to learn new words
  3   As a language learner
  4   I want to see how many answers I got right
  5
  6   Scenario: Successful learning session
  7     Given I have flashcards "apple - яблоко" and "table - стол"
  8     When I answer correctly for both words
  9     Then I should see "2 correct answers and 0 wrong"
 10
```

Фото 3 — сценарий на языке Gherkin

```
features > steps > BDD.py > step_then_result
  1 from behave import given, when, then
  2 from unittest.mock import patch
  3 from flashcards_app import FlashcardApp
  4
  5 @given('I have flashcards "apple - яблоко" and "table - стол"')
  6 def step_given_flashcards(context):
  7     context.app = FlashcardApp(interactive=False)
  8     context.app.add_flashcard("apple", "яблоко")
  9     context.app.add_flashcard("table", "стол")
 10
 11 @when('I answer correctly for both words')
 12 def step_when_correct_answers(context):
 13     # Подменяем input() чтобы вместо пользователя возвращались ответы
 14     with patch('builtins.input', side_effect=["яблоко", "стол"]):
 15         context.result = context.app.start_session(return_result=True)
 16
 17 @then('I should see "2 correct answers and 0 wrong"')
 18 def step_then_result(context):
 19     assert context.result["correct"] == 2, "Expected 2 correct answers"
 20     assert context.result["wrong"] == 0, "Expected 0 wrong answers"
 21
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

  I want to see how many answers I got right
    Then I should see "2 correct answers and 0 wrong" # features/steps/BDD.py:17 0.000s
    Then I should see "2 correct answers and 0 wrong" # features/steps/BDD.py:17
1 feature passed, 0 failed, 0 skipped
1 scenario passed, 0 failed, 0 skipped
3 steps passed, 0 failed, 0 skipped
Took 0min 0.001s
PS C:\Users\artem\Desktop\prog> 
```

Фото 4 — метод BDD

4. SDD (Specification by Example) — Спецификация на примерах

Суть метода:

Требования к системе описываются **на конкретных примерах** (в виде таблиц или наборов данных).

Каждый пример превращается в тест.

Простыми словами:

Вместо длинных описаний — приводим **примеры входных и ожидаемых данных**,

а программа должна вести себя в точности как указано.

Пример таблицы (для приложения с флэш-картами):

Слово	Перевод	Введённый ответ	Ожидаемый результат
sun	солнце	солнце	Верно
moon	луна	луна	Верно
cat	кот	dog	Неверно. Правильный ответ: кот

Зачем:

SDD помогает убедиться, что программа действительно работает **по конкретным случаям из реальной жизни**, а не только по теоретическим требованиям.

```
test_SDD.py > ...
1  import pytest
2  from unittest.mock import patch
3  from flashcards_app import FlashcardApp
4
5  @pytest.mark.parametrize("word,translation,answer,expected_correct", [
6      ("cat", "кошка", "кошка", 1),    # правильный ответ
7      ("dog", "собака", "пес", 0)      # неправильный ответ
8  ])
9  def test_examples(word, translation, answer, expected_correct):
10     app = FlashcardApp(interactive=False)
11     app.add_flashcard(word, translation)
12
13     with patch("builtins.input", side_effect=[answer]):
14         result = app.start_session(return_result=True)
15
16     assert result["correct"] == expected_correct
17     assert result["wrong"] == 1 - expected_correct
18
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\artem\Desktop\prog> py -m pytest -q
..
2 passed in 0.03s
```

Фото 5 — метод SSD

Код программы Приложение для флэш-карт (изучение языков)

```
1 class FlashcardApp:
2     def __init__(self, interactive=True):
3         self.flashcards = [] #хранение кортежей
4         self.interactive = interactive # нужно для тестов, сохраняет состояние пользователем или тестами вызвана
5
6     def add_flashcard(self, word, translation): # добавление новой карточки
7         self.flashcards.append((word, translation)) # добавляем в конец кортежа новую карточку
8         print(f"Карточка '{word}' -> '{translation}' добавлена.")
9
10    def update_flashcard(self, word, new_translation): # обновление перевода
11        for i, (w, t) in enumerate(self.flashcards): # распаковка карточки
12            if w == word:
13                self.flashcards[i] = (word, new_translation)
14                print(f"Перевод для '{word}' обновлён на '{new_translation}'.")
15                return # выходим из функции сразу после нахождения и обновления
16        print(f"Карточка '{word}' не найдена.")
```

Фото 6 — основные функции программы

```
1 def start_session(self, return_result=False):
2     if not self.flashcards:
3         print("Нет карточек для тренировки.")
4         if return_result:
5             return {"correct": 0, "wrong": 0}
6         return
7     correct_count = 0
8     print("\nНачинаем сессию! Введите перевод для каждого слова:")
9
10    for word, correct_translation in self.flashcards:
11        answer = input(f"{word} -> ")
12        if answer.strip().lower() == correct_translation.lower():
13            print("Верно!")
14            correct_count += 1
15        else:
16            print(f"Неверно. Правильный ответ: {correct_translation}")
17
18    wrong_count = len(self.flashcards) - correct_count
19    print(f"\nСессия завершена. Правильных: {correct_count}, Неправильных: {wrong_count}")
20
21    if return_result:
22        return {"correct": correct_count, "wrong": wrong_count}
```

Фото 7 — проверка знания карточек



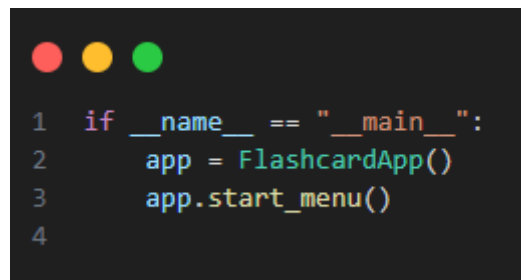
```
1 def show_flashcards(self):
2     if not self.flashcards:
3         print("Карточек нет.")
4         return
5     print("\nСписок карточек:")
6     for w, t in self.flashcards:
7         print(f"{w} -> {t}")
```

Фото 8 — просмотр всех карточек



```
1 def start_menu(self):
2     if not self.interactive:
3         return # отключаем меню для тестов
4     while True:
5         print("\n--- Flashcard App ---")
6         print("1. Добавить карточку")
7         print("2. Обновить карточку")
8         print("3. Показать все карточки")
9         print("4. Провести тренировку")
10        print("5. Выход")
11        choice = input("Выберите действие: ").strip()
12        if choice == "1":
13            word = input("Введите слово: ").strip()
14            translation = input("Введите перевод: ").strip()
15            self.add_flashcard(word, translation)
16        elif choice == "2":
17            word = input("Введите слово для обновления: ").strip()
18            new_translation = input("Введите новый перевод: ").strip()
19            self.update_flashcard(word, new_translation)
20        elif choice == "3":
21            self.show_flashcards()
22        elif choice == "4":
23            self.start_session()
24        elif choice == "5":
25            print("Выход из программы. Пока!")
26            break
27        else:
28            print("Неверный выбор. Попробуйте снова.")
```

Фото 9 — пользовательское меню



```
1  if __name__ == "__main__":
2      app = FlashcardApp()
3      app.start_menu()
4
```

Фото 10 — запуск программы

```

1 class FlashcardApp:
2     def __init__(self, interactive=True):
3         self.flashcards = [] #хранение кортежей
4         self.interactive = interactive # нужно для тестов, сохраняет состояние пользователем или тестами вызвана
5
6     def add_flashcard(self, word, translation): # добавление новой карточки
7         self.flashcards.append((word, translation)) # добавляем в конец кортежа новую карточку
8         print(f"Карточка '{word}' -> '{translation}' добавлена.")
9
10    def update_flashcard(self, word, new_translation): # обновление перевода
11        for i, (w, t) in enumerate(self.flashcards): # распаковка карточки
12            if w == word:
13                self.flashcards[i] = (word, new_translation)
14                print(f"Перевод для '{word}' обновлён на '{new_translation}'.")
15                return # выходим из функции сразу после нахождения и обновления
16        print(f"Карточка '{word}' не найдена.")
17
18    def start_session(self, return_result=False):
19        if not self.flashcards:
20            print("Нет карточек для тренировки.")
21            if return_result:
22                return {"correct": 0, "wrong": 0}
23            return
24        correct_count = 0
25        print("\nНачинаем сессию! Введите перевод для каждого слова:")
26
27        for word, correct_translation in self.flashcards:
28            answer = input(f"{word} -> ")
29            if answer.strip().lower() == correct_translation.lower():
30                print("Верно!")
31                correct_count += 1
32            else:
33                print(f"Неверно. Правильный ответ: {correct_translation}")
34
35        wrong_count = len(self.flashcards) - correct_count
36        print(f"\nСессия завершена. Правильных: {correct_count}, Неправильных: {wrong_count}")
37
38        if return_result:
39            return {"correct": correct_count, "wrong": wrong_count}
40
41    def show_flashcards(self):
42        if not self.flashcards:
43            print("Карточек нет.")
44            return
45        print("\nСписок карточек:")
46        for w, t in self.flashcards:
47            print(f"{w} -> {t}")
48
49    def start_menu(self):
50        if not self.interactive:
51            return # отключаем меню для тестов
52        while True:
53            print("\n--- Flashcard App ---")
54            print("1. Добавить карточку")
55            print("2. Обновить карточку")
56            print("3. Показать все карточки")
57            print("4. Провести тренировку")
58            print("5. Выход")
59            choice = input("Выберите действие: ").strip()
60            if choice == "1":
61                word = input("Введите слово: ").strip()
62                translation = input("Введите перевод: ").strip()
63                self.add_flashcard(word, translation)
64            elif choice == "2":
65                word = input("Введите слово для обновления: ").strip()
66                new_translation = input("Введите новый перевод: ").strip()
67                self.update_flashcard(word, new_translation)
68            elif choice == "3":
69                self.show_flashcards()
70            elif choice == "4":
71                self.start_session()
72            elif choice == "5":
73                print("Выход из программы. Пока!")
74                break
75            else:
76                print("Неверный выбор. Попробуйте снова.")
77
78    if __name__ == "__main__":
79        app = FlashcardApp()
80        app.start_menu()
81

```

Фото 11 – код приложения целиком

Вывод

В ходе выполнения практической работы были изучены и практически применены различные подходы к разработке программного обеспечения, основанные на тестировании: **TDD**, **ATDD**, **BDD** и **SDD**. Каждый из методов продемонстрировал собственные преимущества и особенности применения в процессе создания приложения для флэш-карт.

Метод **TDD** позволил обеспечить высокое качество кода и его устойчивость к ошибкам за счёт предварительного написания тестов. **ATDD** способствовал лучшему пониманию требований со стороны пользователя и формированию прозрачных критериев приёмки. Использование **BDD** сделало процесс тестирования более наглядным и понятным для всех участников разработки, а **SDD** позволил задокументировать функциональность системы в виде конкретных примеров и тестовых случаев.

Применение данных подходов показало, что интеграция тестирования на всех этапах разработки значительно повышает надёжность, предсказуемость и сопровождаемость программного продукта.